

Report conversation

please refactor this code to avoid n+1 queries :

```
class StockCreationBodyModel(BaseModel):
    activation_codes: list[str] | None
    activation_codes_expiration_datetime: datetime | None
    beginning_datetime: datetime | None
    booking_limit_datetime: datetime | None
    price: decimal.Decimal | None
    price_category_id: int | None
    quantity: int | None = Field(None, ge=0,
le=models.Stock.MAX_STOCK_QUANTITY)

def _stock_exists(offer_id: int, stock_payload:
serialization.StockCreationBodyModel) -> bool:
    if not stock_payload.price:
        price_to_check = decimal.Decimal(0)
    else:
        price_to_check = stock_payload.price
    existing_stock = offers_models.Stock.query.filter(
        offers_models.Stock.offerId == offer_id,
        offers_models.Stock.isSoftDeleted == False,
        offers_models.Stock.beginningDatetime ==
stock_payload.beginning_datetime,
        offers_models.Stock.bookingLimitDatetime ==
stock_payload.booking_limit_datetime,
        offers_models.Stock.price == price_to_check,
        offers_models.Stock.priceCategoryId ==
stock_payload.price_category_id,
        offers_models.Stock.quantity ==
stock_payload.quantity,
    ).all()
    return bool(existing_stock)

# ...
for stock_to_create in stocks_to_create:
    if not _stock_exists(body.offer_id,
```

```
stock_to_create):
    # ...
```

To refactor the code and avoid the N+1 query problem, we need to minimize the number of database queries made within a loop. In your current code, the `_stock_exists` function is called inside a loop, which results in a separate database query for each iteration. This is a classic example of an N+1 query issue.

To solve this, we can fetch all relevant stocks in a single query before the loop and then check for the existence of each stock in Python, rather than with separate SQL queries.

Here's how you can refactor your code:

python

 Copy code

```
from typing import List

class StockCreationBodyModel(BaseModel):
    # ... [rest of your model definition]

def _fetch_existing_stocks(offer_id: int, stocks_payload: List[serializati
criteria = [
    offers_models.Stock.offerId == offer_id,
    offers_models.Stock.isSoftDeleted == False,
]
# Create tuples of attributes to filter
stock_attributes = [(stock.price or decimal.Decimal(0),
                    stock.beginning_datetime,
                    stock.booking_limit_datetime,
                    stock.price_category_id,
                    stock.quantity) for stock in stocks_payload]

# Add criteria for each stock attribute
for price, beginning, booking_limit, price_category_id, quantity in stock_attributes:
    criteria.append(
        (offers_models.Stock.price == price) &
        (offers_models.Stock.beginningDatetime == beginning) &
        (offers_models.Stock.bookingLimitDatetime == booking_limit) &
        (offers_models.Stock.priceCategoryId == price_category_id) &
        (offers_models.Stock.quantity == quantity)
    )
```

```

# Perform a single query to fetch all stocks that match any of the cri
return offers_models.Stock.query.filter(or_(*criteria)).all()

def _stock_exists(existing_stocks, stock_payload) -> bool:
    for stock in existing_stocks:
        if (stock.offerId == stock_payload.offer_id and
            stock.price == (stock_payload.price or decimal.Decimal(0)) and
            stock.beginningDatetime == stock_payload.beginning_datetime and
            stock.bookingLimitDatetime == stock_payload.booking_limit_date and
            stock.priceCategoryId == stock_payload.price_category_id and
            stock.quantity == stock_payload.quantity):
            return True
    return False

# ...
existing_stocks = _fetch_existing_stocks(body.offer_id, stocks_to_create)

for stock_to_create in stocks_to_create:
    if not _stock_exists(existing_stocks, stock_to_create):
        # ...

```

This refactoring approach fetches all potentially existing stocks in one query and then checks in-memory. It significantly reduces the number of database queries, especially when the number of stocks in `stocks_to_create` is large.